Name: Bereket Wolderufael
Netid: bgw21001

Multicore CPU Scheduling Simulator → CHRONOS

Project Design Document

1. Introduction

I built a multithreaded CPU scheduling simulator implemented in C++17 that simulates the behaviour of modern operating system schedulers managing multiple CPU cores.

I have implemented four of the CPU scheduling algorithms we covered in class:

- **First-Come-First-Served (FCFS):** processes jobs in arrival order (non-preemptive).
- **Shortest Job First (SJF):** selects the job with the shortest burst time(non-preemptive).
- **Priority Scheduling:** selects jobs based on priority values (non-preemptive).
- **Round Robin (RR):** uses time-slicing with a configurable quantum (preemptive).

2. Objectives

- Functionalities
  - Add CPU scheduling algorithms that model the behavior we covered in class.
  - Supports multi-core execution through parallel worker threads that simulate an independent CPU core.
  - Generate workloads with randomized job parameters (arrival times, burst times, priorities).

- Metrics Collection
  - Track detailed timing metrics for each job, like start time, finish time, waiting time, and turnaround time
  - Calculate aggregate statistics (average waiting time, average turnaround time, and total makespan).
  - Measure system efficiency with CPU utilization percentage and context switch counts.
  - Export data in a CSV format for visualization.

- Visualization and Analysis
    - Generate Gantt charts to show job execution timelines across CPU cores.
    - Create comparison charts to display average metrics across different algorithms.
    - Visualize efficiency metrics like CPU utilization and context switch graphs.
    - Enable algorithm comparison using the compare-all mode.

3. System Design

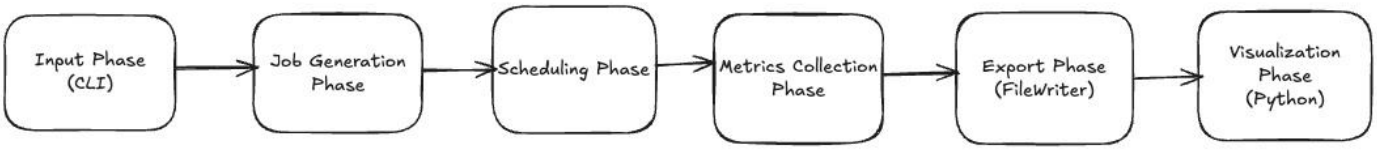
- Architecture Overview

I have followed object-oriented architecture to have a clear separation between scheduling logic, job management, metrics collection, and visualization.

It is organized into the following major components:

```
Chronos Architecture
│
├── main.cpp                    (Entry point and orchestration)
│
├── Core Components
│   ├── Job                     (Process representation)
│   ├── SchedulerEngine         (Main simulation loop)
│   ├── WorkerPool              (Multi-core thread management)
│   └── MetricsCollector        (Performance tracking)
│
├── Scheduling Algorithms
│   ├── ISchedulingPolicy       (Abstract interface)
│   ├── FCFSPolicy              (FCFS implementation)
│   ├── SJFPolicy               (SJF implementation)
│   ├── PriorityPolicy          (Priority implementation)
│   └── RoundRobinPolicy        (RR implementation)
│
├── Utilities
│   ├── CLIParser               (Command-line argument processing)
│   ├── FileWriter              (CSV export functionality)
│   └── AlgorithmComparator     (Compare-all mode handler)
│
└── Visualization
    └── visualize.py            (Matplotlib chart generation)
```

- Data Flow



- Key Design Decisions
  - **Independent Time Tracking Per Core:**
    - Each worker thread maintains **local_core_**time instead of a shared global clock.
      - This eliminates timing conflicts in parallel execution and prevents inflated finish times. Also, I was able to fix a bug where one core's updates affected others.
  - **Compare-All Mode Design:**
    - Generates only summary.csv (aggregate metrics), omitting metrics.csv (per-job details) to prevent massive files with duplicate job IDs.
    - Gantt charts are unavailable in this mode by design(aggregate comparison).
  - **Randomized Job Generation**:
    - Arrival times [0.0, 10.0s]
    - burst times [1.0, 10.0s]
    - priorities [1-5]

Metrics and Calculations:

- All metrics use precise timestamp-based formulas:
  - Waiting Time = start_time - arrival_time (time in ready queue)
  - Turnaround Time = finish_time - arrival_time (total completion time, ≥ burst_time)
  - Makespan = max(finish_time) - min(start_time) (total schedule length)
  - CPU Utilization = total_cpu_time / (makespan × num_cores) (bounded to 0-100%)
  - Context Switches = max(0, num_dispatches - num_cores) (initial dispatches don't count as switches)
    - Example:
      - If we have 2 cores and a 10 second makespan, available capacity = 20 seconds. If total CPU time = 18s, utilization = 90%.

Visualization Pipeline

- Three chart types generated via tools/visualize.py:
    - **Gantt Chart (gantt_chart.png):** Timeline visualization showing job execution periods (solid bars) and waiting periods (hatched bars) across cores. Color-coded by job ID. Available only in single-algorithm mode (requires metrics.csv).
    - **Average Metrics Chart (avg_metrics.png):** Grouped bar chart comparing average waiting time (blue) and turnaround time (coral) across algorithms. Value labels on bars. Always available.
    - **CPU Utilization Chart (utilization.png):** Dual-axis chart with CPU utilization bars (left axis, blue) and context switch line plot (right axis, red). Reveals efficiency vs. overhead trade-offs. Always available.

4. Testing and Verification

Verification Methods:

- **Algorithm correctness:** Manually verified job selection order matches expected behavior (FCFS=arrival order, SJF=shortest burst, etc.)
- **Metrics validation:** Confirmed turnaround ≥ burst, waiting = 0 for immediate starts, utilization ≤ 100% in all cases
- **Multi-core behavior:** Gantt charts showed overlapping execution; finish times matched burst+start calculations
- **Stress testing:** Validated 1-8 cores with 5-100 jobs, and all configurations completed successfully

5. Possible Feature Additions (Future):

- I/O-Bound Process Simulation
    - Current limitation: All jobs are CPU-bound with continuous execution.
    - In the future, I can add I/O blocking periods where jobs voluntarily yield the CPU, implementing I/O wait queues and completion events.

- Trace File Input Support
    - Currently, jobs are randomly generated each run, making exact reproduction difficult. I can later on support reading job specifications from CSV trace files with predefined arrival times, burst times, and priorities.
- Dynamic Priority Adjustment (Aging)

    - Right now, priorities are static throughout a job lifetime, which could potentially cause starvation.
    - I can add a feature to gradually increase the priority of long-waiting jobs to prevent indefinite postponement.

- Preemptive SJF (Shortest Remaining Time First)
    - Can implement SRTF, where a running job is preempted if a shorter job arrives.

- Interactive Visualization
    - Currently, static PNG images are generated after the simulation completes. But I could also create interactive HTML charts for real-time progress monitoring.

6. Lessons I Learned:

- Concurrency Complexity
    - Initially, I had a shared current_time_ across threads that inflated finish times. When Core 1 updated the shared time, it incorrectly affected Core 2's calculations even though they should be independent.
    - Then I added a per-core local time tracking (local_core_time variable in each worker thread).

- Metrics Validation and Accuracy
    - Earlier, the CPU utilization was exceeding 100%, which is physically impossible. Then I noticed a bug in my formula.
        - It wasn't accounting for multiple cores in the utilization calculation (cpu_time / makespan) instead of using cpu_time / (makespan × num_cores).

Note: All the project files came from my repo: https://github.com/Bekjo3/Chronos. I'll also add the future feature points I mentioned here.